

Миграция Apache NiFi с версии 1.26.x на 2.x

Создана пользователем Леликов Александр Александрович. Последнее обновление: 28 авг., 2026 • Время чтения: 10 мин.

- Вводная часть
 - Особенности штатной миграции "ванильного NiFi" и требование версии не ниже 1.27.0
 - Специфика миграции в рамках ПО Arenadata Streaming
- Что входит в подготовку
 - Термины, которые используются при анализе совместимости
 - Removed — удалённый компонент
 - Renamed — переименованный компонент
 - Deprecated — устаревший компонент
 - Changed Configuration — изменённая конфигурация
 - Drop-in replacement — прямая замена
 - Первый этап — определить Process Groups для переноса
 - Второй этап — поиск устаревших процессоров
 - Назначение и принцип работы скрипта-аудитора
 - Целевые процессоры, выявляемые скриптом
 - Формат получаемого первичного отчета
 - После получения списка устаревших процессоров переходим к работе с ними
 - Kafka-процессоры
 - Порядок сохранения и переноса конфигураций Kafka
 - ConvertJSONToSql
 - Controller Services
 - DistributedMapCacheClientService
 - DistributedMapCacheServer
 - Почему Controller Services нужно рассматривать вместе с процессорами
 - ExecuteScript и Jython
 - Parameter Context
 - Что фиксируем в Label
 - Рекомендации по инвентаризации и аудиту потоков перед переносом
 - Локальный чек-лист готовности потока
 - Ссылки:

Вводная часть

Переход с Apache NiFi 1.26.0 на целевую версию NiFi 2.x (рекомендуется не ниже версии 2.5.0) представляет собой мажорное обновление платформы интеграции. Основные архитектурные изменения включают полный отказ от устаревших версионных компонентов, оптимизацию механизмов работы с кэшем и базами данных, переход на нативный Python 3.x, а также обновление требований к среде выполнения (обязательное использование Java 21).

В процессе перевода потоков данных с NiFi 1.x на NiFi 2.x компоненты, выведенные из эксплуатации, не смогут корректно отобразить свои настройки и будут представлены на canvas без конфигурации (**Invalid**). Для предотвращения потери конфигурационных данных и обеспечения непрерывности процессов интеграции требуется проведение предварительного аудита и подготовка потоков.

Особенности штатной миграции "ванильного NiFi" и требование версии не ниже 1.27.0

Согласно официальной документации разработчика (Apache NiFi Migration Guidance), классический сценарий обновления сервера предусматривает обязательный промежуточный шаг:

NiFi 1.26.0 -> NiFi (1.27.0) 1.28.1 -> NiFi 2.x.0

С чем связано это требование:

1. **Смена формата хранения потока:** В версии NiFi 1.x системное состояние холста хранилось в формате XML (flow.xml.gz), тогда как в NiFi 2.x произошел переход на JSON (flow.json.gz).
2. **Встроенные механизмы миграции:** Начиная с версии 1.27.0, разработчики NiFi добавили переходные модули и методы автоконвертации структур, которые при перезаписи потока приводят его к схеме, совместимой с загрузчиком версии 2.x.
3. **Автоматическое переименование компонентов (Renamed):** В версии 1.27+ заложены правила подмены системных классов при первом старте (например, автоматическая трансляция служб DistributedMapCacheServer -> MapCacheServer и DistributedMapCacheClientService -> MapCacheClientService).

При прямом обновлении сервера с версии 1.26.0 на 2.x без промежуточного шага встроенный конвертер NiFi не сможет корректно прочитать устаревшие XML-структуры.

Специфика миграции в рамках ПО Arenadata Streaming

Переход с Apache NiFi 1.x на NiFi 2.5. в нашем случае нельзя рассматривать как обычное, штатное обновление версии NiFi.

В Arenadata Streaming (ADS) сервис NiFi2 поставляется как отдельный изолированный продукт. Например, в **ADS 3.9.1.1** появился отдельный сервис **NiFi2 2.5.0_arenadata1**, при этом NiFi 1.x в той же версии ADS продолжил работать на NiFi **1.28.1_arenadata2**. В документации ADS прямо указано, что NiFi2 устанавливается в отдельный кластер вместе с ZooKeeper и требует среды Java 21.

Поэтому для разработчиков потоков задача выглядит не как обновление одного сервера через промежуточные версии, а как **подготовка существующих потоков NiFi 1.26.0 к последующему переносу в отдельный целевой экземпляр NiFi 2.x.0**.

Влияние на процесс разработки:

- Мы не зависим от автоматической конвертации полного системного файла flow.xml.gz сервером.
- Перенос осуществляется на уровне отдельных групп процессов (Process Groups) через экспорт/импорт определений потоков (JSON Flow Definition), либо через версионирование (NiFi Registry).
- Существующий поток остаётся неизменным в исходном экземпляре, а его процессоры и службы инвентаризируются, адаптируются и подготавливаются для загрузки в целевой NiFi 2.x.0.

В этой статье рассматривается именно **подготовка потоков к переносу между инстансами NiFi v1.x → v2.5.x**. Статья носит рекомендательный характер.

Что входит в подготовку

Подготовка состоит из нескольких независимых задач:

1. определить перечень потоков, которые действительно нужно переносить;
2. найти в этих потоках компоненты, несовместимые с NiFi 2.5.0 и выше;
3. определить, какие компоненты требуют замены, а какие могут быть автоматически мигрированы;
4. сохранить настройки компонентов, которые нельзя потерять при замене;
5. проверить Controller Services;
6. проверить ExecuteScript и наличие Jython на текущем инстансе NiFi;
7. подготовить Parameter Contexts;
8. определить способ последующего переноса;
9. (опционально) сформировать итоговый перечень работ для каждого потока.

Цель подготовительного этапа - не изменить все потоки заранее, а **получить понятный и проверяемый план работ**.

Термины, которые используются при анализе совместимости

При подготовке к миграции потоков данных необходимо классифицировать компоненты по типам изменений. В соответствии с официальным руководством по миграции разработчиков Apache NiFi (раздел Migration Guidance на wiki.apache.org), все изменения делится на следующие категории:

Removed — удалённый компонент

Компонент полностью отсутствует в целевой версии NiFi.

Использовать его в NiFi 2.5.0 нельзя.

Такой компонент требует:

- ручной замены;
- перестройки части потока;
- либо принятия другого архитектурного решения.

Например:

```
ConvertJSONToSql
```

относится к компонентам, удалённым из версий NiFi 2.0. В официальном списке для него не указана прямая замена.

Поэтому его нельзя рассматривать как обычное переименование одного процессора в другой.

Renamed — переименованный компонент

Компонент получил новое имя, но его функциональность сохраняется, а NiFi предусматривает автоматическую миграцию.

Например:

```
DistributedMapCacheClientService
    ↓
MapCacheClientService
и
DistributedMapCacheServer
    ↓
MapCacheServer
```

Apache NiFi указывает эти изменения именно как переименование при миграции в NiFi 2.0.

Поэтому наличие такого компонента в исходном потоке само по себе ещё не означает, что его необходимо вручную пересоздавать.

Но его необходимо обнаружить и проверить после переноса.

Deprecated — устаревший компонент

Компонент объявлен устаревшим и предназначен для удаления в дальнейшем.

Deprecated не означает автоматически *Removed* в текущей целевой версии.

Тем не менее наличие такого компонента в потоке является сигналом:

> необходимо проверить, существует ли он в целевой версии NiFi и какой вариант рекомендуется использовать вместо него.

Официальная документация Apache NiFi содержит отдельный перечень deprecated-компонентов и указывает для многих из них рекомендуемые альтернативы.

Changed Configuration — изменённая конфигурация

Сам компонент продолжает существовать, но его свойства, способы настройки или зависимости изменились.

В этом случае процессор может присутствовать в целевой версии, однако его конфигурация может потребовать ручной проверки.

Это особенно важно для компонентов, работающих с внешними системами и Controller Services.

Drop-in replacement — прямая замена

Drop-in replacement — компонент, который можно использовать вместо старого с минимальными изменениями в потоке.

Например, если старый компонент выполнял ту же функцию, а его настройки и интерфейс практически совпадают с новым.

Первый этап — определить Process Groups для переноса

На главном холсте (Root Canvas) экземпляра NiFi 1.x.0 группы процессов верхнего уровня чаще всего организованы в виде контейнеров по ландшафтам, контурам или бизнес-доменам (например, ATHENA, RETAIL_FD, RATES, WAY4_ATN). Каждая такая верхнеуровневая группа представляет собой не единичный процесс, а целое дерево, содержащее множество интеграционных потоков (Flows), распределенных по вложенным дочерним Process Groups.

По этому, до анализа компонентов целесообразно определить, какие верхнеуровневые Process Groups должны попасть в целевой NiFi 2.5.0.

Для каждого Process Group необходимо определить:

- является ли она актуальной и находящейся в промышленной эксплуатации;
- находится ли она в стадии разработки;
- переносится ли группа в целевую среду NiFi 2.x;
- подлежит ли она выводу из эксплуатации и удалению вместо переноса.

Для каждой верхнеуровневой Process Group желательно зафиксировать:

Process Group	Перенос	Комментарий
ATHENA	Да	Актуальный
RETAIL_FD	Да	Актуальный
RATES	Нет	В Разработке

Такой подход позволяет не тратить ресурсы команды на аудит и адаптацию компонентов тех потоков, которые не требуются в целевой среде. Кроме того, предварительная классификация позволяет зафиксировать итоговый список миграции в рамках **ADR (Architecture Decision Record)**: формализовать решение об отказе от переноса неактуальных контуров, закрепить правила вывода устаревших процессов из эксплуатации и утвердить точный перечень переносимых Process Group и потоков.

Второй этап — поиск устаревших процессоров

После того как определен целевой перечень групп процессов (Process Groups), выполняется этап технической инвентаризации.

Поскольку ручной визуальный обход холста на разветвленных инстансах неэффективен и сопряжен с рисками пропуска несовместимых компонентов, аудит автоматизируется с помощью специализированного скрипта на **Groovy**.

Назначение и принцип работы скрипта-аудитора

Скрипт разворачивается и запускается непосредственно в **исходном экземпляре NiFi 1.x (на источнике)**.

Ключевые особенности механизма:

1. **Рекурсивный обход REST API:** Скрипт обращается к локальному API инстанса (`/nifi-api`), авторизуется по JWT-токену и обходит все уровни вложенности Process Groups, начиная от Root.

2. **Гибкая подстрочная фильтрация:** Поиск выполняется регистронезависимо (`toLowerCase()`) как по сигнатуре Java-класса (`type`), так и по пользовательскому названию компонента на холсте (`name`). Это позволяет гарантированно обнаружить процессор даже при наличии суффиксов версий бандлов NAR (например, `org.apache.nifi.processors.standard.ConvertJSONToSql-1.26.0`) или при его частичном переименовании.
3. **Генерация человекочитаемого отчета:** Результатом выполнения скрипта является входящий в очередь `FlowFile` с текстовым отчетом, содержащим точные пути («хлебные крошки» / `Breadcrumbs`) на `canvas`, уникальные ID и статусы каждого проблемного компонента.

Целевые процессоры, выявляемые скриптом

Скрипт сфокусирован на поиске компонентов, которые **физически удалены из ядра NiFi 2.x** и при попытке переноса гарантированно станут невалидными пустышками (*Ghost / Missing Components*):

- **Версионные процессоры Kafka:** `ConsumeKafka_2_6`, `PublishKafka_2_6`, `ConsumeKafkaRecord_2_6`, `PublishKafkaRecord_2_6` (подлежат замене на универсальные процессоры + `Kafka3ConnectionService`).
- **Генераторы прямого SQL:** `ConvertJSONToSql` (прямой альтернативы нет, требует перестройки логики на Record-ориентированную цепочку `ConvertRecord -> PutDatabaseRecord`).

Формат получаемого первичного отчета

После единичного запуска скрипта формируется структурированный отчет следующего вида:

```
=====
      ОТЧЕТ ОБ АУДИТЕ КОМПОНЕНТОВ ДЛЯ МИГРАЦИИ В NIFI 2.X
=====
Дата проверки: 2026-08-25 12:16:28
Пользователь: alex
Всего найдено заменяемых компонентов: 14

[1] Процессор: 'PublishKafka_2_6'
    ID:      4d7c3d18-54b3-3ab5-5cf1-7981ad5cd46e
    Тип:     org.apache.nifi.processors.kafka.pubsub.PublishKafka_2_6
    Статус:  STOPPED
    Путь:    NiFi Flow -> KAFKA_LOGS -> PublishKafka_2_6
-----

[2] Процессор: 'ConvertJSONToSQL'
    ID:      efdca104-019f-1000-cda8-c0e372dbdc30
    Тип:     org.apache.nifi.processors.standard.ConvertJSONToSQL
    Статус:  STOPPED
    Путь:    NiFi Flow -> Conver1 -> Conver2 -> Conver3 -> Conver4 -> ConvertJSONToSQL
-----

[3] Процессор: 'PublishKafkaRecord_2_6'
    ID:      100611ba-17ac-123c-2624-67f214717f1e
    Тип:     org.apache.nifi.processors.kafka.pubsub.PublishKafkaRecord_2_6
    Статус:  STOPPED
    Путь:    NiFi Flow -> kafka -> PublishKafkaRecord_2_6
-----

[4] Процессор: 'ConsumeKafkaRecord_2_6'
    ID:      100611b9-17ac-123c-4346-7999ebc9c93a
    Тип:     org.apache.nifi.processors.kafka.pubsub.ConsumeKafkaRecord_2_6
    Статус:  STOPPED
    Путь:    NiFi Flow -> kafka -> ConsumeKafkaRecord_2_6
-----
```

Для работы процессора выполняющего рекурсивный поиск устаревших процессоров необходимо добавить пользовательские свойства:

- `nifi.api.url` - url nifi-api,
- `nifi.api.user` - user name,
- `nifi.api.password` - user password (поддерживает Parameter Contexts).

Исходный код Groovy-скрипта аудита: `script.groovy`

После получения списка устаревших процессоров переходим к работе с ними

Kafka-процессоры

В Apache NiFi 2.x изменён подход к интеграции с Apache Kafka: платформенный код полностью отказался от специфичных версионных суффиксов в названиях процессоров (таких как `_1_0`, `_2_0`, `_2_6`)

В ветке 2.x используются универсальные базовые процессоры, а все параметры подключения, аутентификации и работы с кластером Kafka вынесены в отдельный **Controller Service** — `Kafka3ConnectionService`.

Матрица замен компонентов Kafka

В соответствии с официальным реестром компонентов Apache NiFi, старые версии процессоров заменяются по следующей схеме:

- `ConsumeKafka_1_0` / `ConsumeKafka_2_0` / `ConsumeKafka_2_6` -> **ConsumeKafka + Kafka3ConnectionService**
- `PublishKafka_1_0` / `PublishKafka_2_0` / `PublishKafka_2_6` -> **PublishKafka + Kafka3ConnectionService**
- `ConsumeKafkaRecord_1_0` / `ConsumeKafkaRecord_2_0` / `ConsumeKafkaRecord_2_6` -> **ConsumeKafkaRecord + Kafka3ConnectionService**
- `PublishKafkaRecord_1_0` / `PublishKafkaRecord_2_0` / `PublishKafkaRecord_2_6` -> **PublishKafkaRecord + Kafka3ConnectionService**

Порядок сохранения и переноса конфигураций Kafka

Поскольку при переходе на NiFi 2.x часть настроек процессора мигрирует в службу **Kafka3ConnectionService**, перед удалением или заменой исходного компонента разработчик должен зафиксировать его текущую конфигурацию.

Для этого непосредственно на холсте (Canvas) рядом с заменяемым процессором создаётся текстовая заметка (Label), играющая роль страховочного "снимка" параметров.

Минимальный набор параметров для фиксации:

Имя топика (Topic Name);
Идентификатор группы потребителей (Group ID / Application ID);
Список серверов брокеров (Bootstrap Servers);
Параметры безопасности (Security Protocol, SASL Mechanism);
Настройки SSL и связанные службы (SSL Context Service);
Связанные службы обработки записей (Record Reader, Record Writer).

После переноса схемы в NiFi 2.x указанные в Label параметры используются для настройки целевого процессора и создания/конфигурирования службы **Kafka3ConnectionService**.

ConvertJSONToSql

Этот процессор необходимо считать отдельным случаем.

В официальном списке Apache NiFi он обозначен как удалённый (Removed) компонент, причём прямой альтернативы не указано.

Поэтому для него нельзя использовать простой сценарий:

старый процессор



новый процессор

Здесь необходимо заранее определить, какая часть логики выполнялась ConvertJSONToSql, и каким набором актуальных процессоров она будет реализована в целевой системе.

Это уже не механическая замена компонента, а **перестройка** части потока.

На подготовительном этапе достаточно:

1. найти все экземпляры ConvertJSONToSql;
2. зафиксировать их расположение;
3. определить, что именно они делают;
4. сохранить необходимые настройки;
5. включить их в отдельный список ручной переработки.

Controller Services

Controller Services проверяются отдельно от процессоров.

Обычно используется небольшое количество таких служб, поэтому для них не требуется отдельный автоматический сканер.

Проверка выполняется вручную.

Основное внимание необходимо уделить службам кэширования.

DistributedMapCacheClientService

В NiFi 2.x:

```
DistributedMapCacheClientService
    ↓
MapCacheClientService
```

DistributedMapCacheServer

В NiFi 2.x:

```
DistributedMapCacheServer
    ↓
MapCacheServer
```

Это пример категории Renamed, а не просто Removed. Apache NiFi прямо описывает эти изменения как переименование при миграции в 2.0.

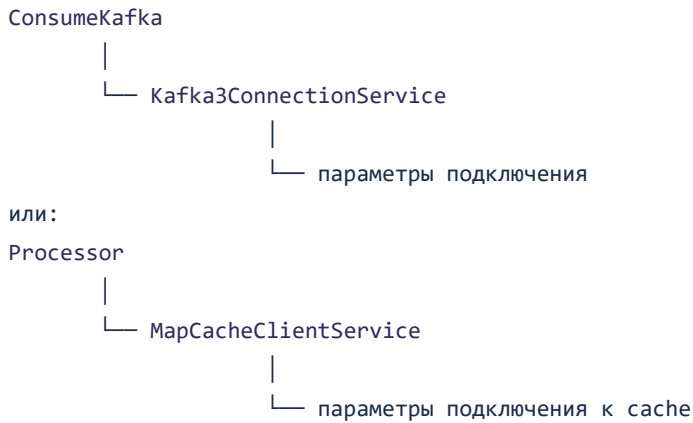
Поэтому здесь задача подготовки состоит в том, чтобы:

1. найти используемые DistributedMapCache;
2. определить, какие Process Groups от них зависят;
3. зафиксировать настройки;
4. проверить соответствующий компонент в NiFi 2.X.0;
5. после переноса убедиться, что зависимости процессоров восстановлены.

Почему Controller Services нужно рассматривать вместе с процессорами

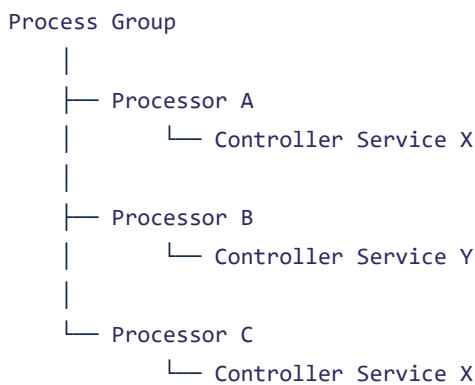
Процессор сам по себе не всегда содержит всю необходимую конфигурацию.

Например:



Поэтому недостаточно получить список процессоров.

Нужно понимать зависимости:



Именно поэтому Controller Services должны быть проверены до начала переноса.

ExecuteScript и Jython

Отдельно необходимо проверить процессоры:

```
ExecuteScript
```

Сам **ExecuteScript** не является причиной миграции.

Проблема возникает, если он использует:

Jython

Jython был объявлен **deprecated**, а затем удалён. Apache указывает для него в качестве направлений миграции Groovy или Native Python в NiFi 2.0.

Поэтому необходимо вручную найти:

```
ExecuteScript
  Script Engine = python
```

и определить дальнейшую судьбу каждого такого скрипта.

На первом этапе удобнее создать список:

```
Process Group
↓
ExecuteScript
```

↓
Jython

После чего каждый такой случай рассматривается отдельно.

Parameter Context

Parameter Context необходимо рассматривать как самостоятельную часть потока.

В Process Group / поток могут входить:

- *Processor*
- *Controller Services*
- *Parameter Context*

и эти объекты должны быть согласованы между исходным и целевым экземплярами NiFi.

В зависимости от выбранного способа переноса Parameter Context может попасть в целевой экземпляр вместе с экспортируемым элементом либо отдельно.

Поэтому на подготовительном этапе необходимо определить:

- какие Process Groups переносятся
- какие Parameter Context к ним привязаны.

Чувствительные значения / Sensitive Parameters - **НЕ ПЕРЕНОСЯТСЯ!**

Что фиксируем в Label

Label нужен не для того, чтобы заменить систему конфигурации NiFi.

Его задача намного проще:

> сохранить рядом со старым компонентом информацию, которая понадобится при ручной замене или проверке.

Например:

```
OLD PROCESSOR
ConsumeKafka_2_6
Topic: customer.events
Group ID: customer-loaderBootstrap
Servers: kafka01:9092,kafka02:9092
SSL: SSLContextServiceRecord
Reader: JsonTreeReader
Для чувствительных параметров:
Password: <SENSITIVE>
```

Таким образом, после переноса разработчик может быстро восстановить исходную конфигурацию, не возвращаясь к старому экземпляру только для того, чтобы посмотреть настройки.

Рекомендации по инвентаризации и аудиту потоков перед переносом

В прикладной разработке архитектурной единицей является сквозной интеграционный поток (Flow) — цепочка от источника через узлы обработки к целевой системе, которая может состоять из одной или нескольких связанных Process Groups.

Для разработчиков оценка объема изменений строится на сквозном анализе жизненного цикла данных внутри потока.

Локальный чек-лист готовности потока

Перед подготовкой потока к экспорту рекомендуется проверить его по четырем функциональным слоям:

1. Слой приема данных (Источники):

- Наличие версионных процессоров Kafka (ConsumeKafka_, ConsumeKafkaRecord_), требующих замены на универсальные процессоры + Kafka3ConnectionService.
- Наличие устаревших механизмов поллинга/чтения БД или веб-запросов (например, прямые вызовы прокси без StandardProxyConfigurationService).

2. Слой обработки и трансформации (Бизнес-логика):

- Наличие процессоров ConvertJSONToSql (требуется перепроектирование логики на Record-цепочку).
- Проверка скриптов в ExecuteScript на использование движка Jython (Script Engine = python) -> принятие решения о переводе на Groovy или замене.
- (Рекомендовано) Наличие прямых SQL-генераторов и PutSQL -> оценка перехода на батчевую запись через PutDatabaseRecord.

3. Слой целевой выгрузки (Приемники):

- Проверка публикации в Kafka (PublishKafka_ -> PublishKafka + Kafka3ConnectionService).
- Проверка целевых подключений к СУБД, кэшам и внешним хранилищам.

4. Сквозная инфраструктура и окружение потока:

- Controller Services потока: инвентаризация зависимостей от служб кэша (DistributedMapCache -> MapCacheServer), пулов соединений (DBCPConnectionPool) и реестров схем (SchemaRegistry).
- Параметры и секреты: проверка привязанных Parameter Contexts потока, отсутствие устаревших локальных переменных Variable Registry и фиксация списка Sensitive Parameters, требующих ручного ввода после переноса.

Ссылки:

Migration Guidance

https://cwiki.apache.org/confluence/spaces/NIFI/pages/57905503/Migration+Guidance?_x_tr_hist=true

Deprecated Components and Features

<https://cwiki.apache.org/confluence/spaces/NIFI/pages/235834208/Deprecated+Components+and+Features>

Нет меток